

Quiz Progress Tracking — State Machine Concept

This document contains the prototype Python implementation used for the Quiz Progress Tracking and Report research task. The code that follows implements a small state-machine style flow for a quiz session: it shows questions, waits for an answer, prompts the learner to speak when the correct choice is selected, simulates recording and evaluation, displays feedback, persists session events and aggregates, and finally exports session events and per-word aggregates to CSV. The implementation uses a SessionContext JSON file to persist the current state and minimal context (current question index, attempts left, aggregates, and raw events) so the session can be resumed (history behavior). The code maps states such as QuestionDisplayed, AwaitingAnswer, AwaitingSpeech, Evaluating, Feedback, Persisting, and ShowingSummary to explicit string-state values and performs entry/exit side-effects (e.g., start_recording, invoke_evaluator, persist_event) accordingly.

Prototype Code (Python)

```
#!/usr/bin/env python3
#!/usr/bin/env python3
"""
QUIZ PROGRESS - Prototype implementation

HOW TO USE (included as comments for quick reference):
- Download this file (quiz_progress_tracking.py) and run it with Python 3.8+:
  python3 quiz_progress_tracking.py

- What it does:
  * Implements a prototype state machine: question display, answer selection,
    speech prompt/recording, evaluation, feedback, persistence, summary and
    CSV export.
  * Persists a minimal SessionContext to JSON (session_context_demo.json) to
    demonstrate history/resume behavior.
  * Simulates an STT/prosody evaluator (invoke_evaluator stub). Replace that
    stub with a real evaluator when integrating.
  * Exports two CSV files on demo run:
    - demo_session_events.csv (raw event log)
    - demo_session_aggregates.csv (per-word aggregates)

- Replacing the simulated evaluator:
  * Find the function `invoke_evaluator(self, filepath)` and replace its body
    with a call to your STT/prosody
    service (synchronous or async). The function must return a dict:
    {"transcript": "<text>", "confidence": <float 0.0-1.0>, "stress_ok":
    <bool>}
    The state machine uses `confidence` and `stress_ok` to compute the
    verdict.

- Persistence and resume:
  * SessionContext is saved to `session_context_demo.json` (or the path
    passed to the machine).
    The machine restores `current_state` and `current_index` on load; use
    those to re-enter the proper state.
  * On restore, be conservative about auto-resuming audio: prefer to prompt
    the user before restarting recording.
```

- Output CSVs:
 - * demo_session_events.csv: columns (question_index, word, verdict, audio_path, timestamp)
 - * demo_session_aggregates.csv: columns (word, times_practiced, correct_count, total_attempts, avg_confidence)
- Integrating into Flutter/Dart:
 - * Use this script as a behavioral spec. Map state names to Dart enums (e.g., InSession_AwaitingSpeech).
 - * Implement entry/exit actions (start_recording, invoke_evaluator, persist_event) as side-effects in Bloc/Riverpod.
 - * Persist a SessionContext similar to this script (JSON schema) using Hive or sqflite.
- Testing and next steps:
 - * Add unit tests for the three traced scenarios: normal flow, interrupted+resume, export failure+retry.
 - * Replace simulated evaluator with a real STT/prosody component or a local heuristic (F0/stress).
 - * Optionally adapt this python module to run headless tests for the Dart implementation to validate logic.

End of HOW TO USE notes.
 """

```
import json, time, csv, os, random
from typing import List, Dict, Any
```

```
SESSION_FILE = "session_context.json"
```

```
class SessionContext:
    def __init__(self, questions: List[Dict[str, Any]] = None):
        self.questions = questions or []
        self.current_index = 0
        self.attempts_left = 2
        self.aggregates = {}
        self.raw_events = []
        self.current_state = "Idle"
        self.partial_audio = None

    def to_dict(self):
        return {
            "questions": self.questions,
            "current_index": self.current_index,
            "attempts_left": self.attempts_left,
            "aggregates": self.aggregates,
            "raw_events": self.raw_events,
            "current_state": self.current_state,
            "partial_audio": self.partial_audio
        }

    @classmethod
    def from_dict(cls, d):
        ctx = cls(d.get("questions", []))
        ctx.current_index = d.get("current_index", 0)
        ctx.attempts_left = d.get("attempts_left", 2)
        ctx.aggregates = d.get("aggregates", {})
        ctx.raw_events = d.get("raw_events", [])
        ctx.current_state = d.get("current_state", "Idle")
        ctx.partial_audio = d.get("partial_audio", None)
        return ctx
```

```

def save(self, path=SESSION_FILE):
    with open(path, "w", encoding="utf8") as f:
        json.dump(self.to_dict(), f, indent=2)
        f.flush()

@classmethod
def load(cls, path=SESSION_FILE):
    if not os.path.exists(path):
        return None
    with open(path, "r", encoding="utf8") as f:
        d = json.load(f)
    return cls.from_dict(d)

class QuizStateMachine:
    def __init__(self, context: SessionContext = None,
persistence_path=SESSION_FILE):
        self.context = context or SessionContext([])
        self.state = self.context.current_state or "Idle"
        self.persistence_path = persistence_path
        self.is_online = True

    def persist_context(self):
        self.context.current_state = self.state
        self.context.save(self.persistence_path)
        print(f"[persist] state={self.state}, idx={self.context.current_index},
attempts_left={self.context.attempts_left}")

    def load_context(self):
        ctx = SessionContext.load(self.persistence_path)
        if ctx:
            self.context = ctx
            self.state = ctx.current_state or "Idle"
            print(f"[load] Restored state={self.state},
idx={self.context.current_index}")
            return True
        return False

    def start_session(self, questions):
        print("[event] start_session")
        self.context.questions = questions
        self.context.current_index = 0
        self.context.attempts_left = 2
        self.context.raw_events = []
        self.context.aggregates = {}
        self.state = "LoadingSession"
        time.sleep(0.02)
        self.on_session_loaded()

    def on_session_loaded(self):
        print("[event] session_loaded")
        self.state = "InSession_QuestionDisplayed"
        self.persist_context()
        self.show_question()

    def show_question(self):
        if self.context.current_index >= len(self.context.questions):
            self.complete_session()
            return
        q = self.context.questions[self.context.current_index]
        print(f"[state] QuestionDisplayed index={self.context.current_index}
word={q.get('word')}")
        self.state = "InSession_AwaitingAnswer"
        self.persist_context()

```

```

def user_selects_choice(self, choice_id: int):
    print(f"[event] user_selects_choice choice_id={choice_id}")
    if self.state != "InSession_AwaitingAnswer":
        print(" Ignored: not awaiting answer")
        return
    q = self.context.questions[self.context.current_index]
    correct_id = q.get("correct_index", 0)
    if choice_id == correct_id:
        print(" Choice is CORRECT")
        if self.context.attempts_left > 0:
            self.prompt_speak()
            self.state = "InSession_AwaitingSpeech"
            self.persist_context()
            self.start_recording()
        else:
            print(" No attempts left -> Feedback")
            self.state = "InSession_Feedback"
            self.show_feedback("no_attempts")
    else:
        print(" Choice is INCORRECT -> Feedback")
        self.state = "InSession_Feedback"
        self.show_feedback("incorrect")

def prompt_speak(self):
    print(" action: prompt_speak() -> Please say the word now.")

def start_recording(self):
    print(" action: start_recording() (simulated)")
    filepath = f"/tmp/rec_{int(time.time()*1000)}.wav"
    time.sleep(0.02)
    self.record_complete(filepath)

def record_complete(self, filepath):
    print(f"[event] record_complete filepath={filepath}")
    if self.state != "InSession_AwaitingSpeech":
        print(" Ignored: not awaiting speech")
        return
    self.state = "InSession_Evaluating"
    self.persist_context()
    result = self.invoke_evaluator(filepath)
    self.on_stt_result(result, filepath)

def invoke_evaluator(self, filepath):
    word = self.context.questions[self.context.current_index].get("word",
"unknown")
    seed = abs(hash(word)) % 1000
    random.seed(seed + int(time.time()) % 5)
    confidence = round(random.uniform(0.6, 0.98), 2)
    stress_ok = confidence > 0.75
    transcript = word if stress_ok else word[:max(1, len(word)-1)]
    print(f" invoke_evaluator -> transcript='{transcript}',
confidence={confidence}, stress_ok={stress_ok}")
    return {"transcript": transcript, "confidence": confidence,
"stress_ok": stress_ok}

def on_stt_result(self, result, filepath):
    print(f"[event] stt_result confidence={result.get('confidence')}")
    conf = result.get("confidence", 0.0)
    stress_ok = result.get("stress_ok", False)
    if conf >= 0.8 and stress_ok:
        verdict = "correct"
    elif conf < 0.5:

```

```

        verdict = "low_confidence"
    else:
        verdict = "close"
    self.state = "InSession_Feedback"
    self.show_feedback(verdict, result, filepath)
    self.persist_event(verdict, result, filepath)

def show_feedback(self, verdict, result=None, filepath=None):
    print(f"[state] Feedback -> verdict={verdict}")
    if verdict == "correct":
        print("    Feedback: Good job!")
    elif verdict == "close":
        print("    Feedback: Close - try again for better stress/pitch.")
    elif verdict == "low_confidence":
        print("    Feedback: Low confidence; please re-record.")
    elif verdict == "incorrect":
        print("    Feedback: That choice was incorrect. Try next time.")
    elif verdict == "no_attempts":
        print("    Feedback: No attempts left for this word. Moving on.")

def persist_event(self, verdict, result, filepath):
    print("[action] persist_event()")
    q = self.context.questions[self.context.current_index]
    word = q.get("word", "unknown")
    evt = {
        "question_index": self.context.current_index,
        "word": word,
        "verdict": verdict,
        "result": result,
        "audio_path": filepath,
        "timestamp": int(time.time())
    }
    self.context.raw_events.append(evt)
    ag = self.context.aggregates.setdefault(word, {
        "times_practiced": 0,
        "correct_count": 0,
        "total_attempts": 0,
        "avg_confidence": 0.0
    })
    ag["times_practiced"] += 1
    ag["total_attempts"] += 1
    if verdict == "correct":
        ag["correct_count"] += 1
    prev_avg = ag.get("avg_confidence", 0.0)
    ag["avg_confidence"] = round(((prev_avg * (ag["times_practiced"] - 1))
+ (result.get("confidence", 0.0))) / ag["times_practiced"], 2)
    self.context.current_index += 1
    self.context.attempts_left = 2
    self.state = "InSession_Persisting"
    self.persist_context()
    if self.context.current_index >= len(self.context.questions):
        self.complete_session()
    else:
        self.state = "InSession_QuestionDisplayed"
        self.show_question()

def complete_session(self):
    print("[event] complete_session -> ShowingSummary")
    self.state = "ShowingSummary"
    self.persist_context()

def export_csv(self, out_path="session_events.csv",
aggregates_path="session_aggregates.csv"):

```

```

        print(f"[action] export_csv out_path={out_path}")
        with open(out_path, "w", newline='', encoding="utf8") as csvfile:
            writer = csv.DictWriter(csvfile,
                                    fieldnames=["question_index", "word", "verdict", "audio_path", "timestamp"])
            writer.writeheader()
            for e in self.context.raw_events:
                writer.writerow({
                    "question_index": e.get("question_index"),
                    "word": e.get("word"),
                    "verdict": e.get("verdict"),
                    "audio_path": e.get("audio_path"),
                    "timestamp": e.get("timestamp")
                })
        with open(aggregates_path, "w", newline='', encoding="utf8") as csvfile:
            writer = csv.DictWriter(csvfile,
                                    fieldnames=["word", "times_practiced", "correct_count", "total_attempts", "avg_confidence"])
            writer.writeheader()
            for w, ag in self.context.aggregates.items():
                writer.writerow({
                    "word": w,
                    "times_practiced": ag.get("times_practiced", 0),
                    "correct_count": ag.get("correct_count", 0),
                    "total_attempts": ag.get("total_attempts", 0),
                    "avg_confidence": ag.get("avg_confidence", 0.0)
                })
        print(" export complete")
        return out_path, aggregates_path

def demo_run():
    manifest = [
        {"word": "banana", "correct_index": 0},
        {"word": "software", "correct_index": 1},
        {"word": "computer", "correct_index": 2}
    ]
    ctx = SessionContext(manifest)
    sm = QuizStateMachine(ctx, persistence_path="session_context_demo.json")
    sm.start_session(manifest)
    for i in range(len(manifest)):
        current = sm.context.questions[sm.context.current_index]
        correct_choice = current["correct_index"]
        sm.user_selects_choice(correct_choice)
        time.sleep(0.02)
    if sm.state == "ShowingSummary":
        sm.export_csv(out_path="demo_session_events.csv",
                      aggregates_path="demo_session_aggregates.csv")

if __name__ == "__main__":
    demo_run()

```